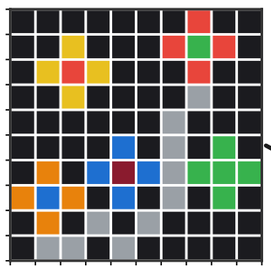
Per-task test-time training, served on one 16 GB GPU

Train a LoRA in PyTorch, bridge it to GGUF, hot-swap it on llama.cpp, then vote across augmentations.



ARC task

train pairs + test input

Per-task TTT: the serving bridge

Fit LoRA

PyTorch · rank-16
leave-one-out · ~20 steps @ 2s
trains with random recolour

adapter

Convert → GGUF

convert_lora_to_gguf.py

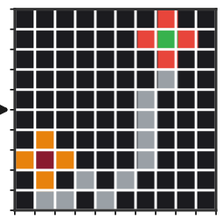
GGUF

Serve · llama.cpp

base Q8_0 + hot-swap
POST /lora-adapters
~115 tok/s decode

AIRV decoding

8 D4 augmentations
transform → generate → invert
vote → top-2



answer

2 attempts

Why the bridge is the contribution

PyTorch can train the adapter but decodes at only 13 tok/s, far too slow to run per-task TTT across the eval set; llama.cpp serves at ~115 tok/s with the adapter but cannot train it. Converting to GGUF and hot-swapping links the two. Train, convert, and serve run as three phases so they never share the 16 GB at once.

Model: NVARC Qwen3-4B (16-token grid vocab), Q8_0 GGUF 3.6 GB Hardware: 1× RTX 5070 Ti, 16 GB, Blackwell, Windows